Andy, Yu-Guang Chen

EE6094 CAD for VLSI Design
Programming Assignment 1: Benchmark Translator
(Due: 23:59:59, 2025/03/20)

-v1.0 20250220

Introduction

When evaluating the quality of various algorithms in Computer-Aided Design (CAD), it is crucial to employ a set of representative circuits for a fair comparison. These circuits should encompass a broad range of behaviors and structures found in different types of Integrated Circuits (ICs) to ensure credible comparisons. The ISCAS'85 benchmark [1] consists of ten combinational networks originally provided to authors at the 1985 International Symposium on Circuits And Systems (ISCAS). The ISCAS'85 netlist format was never formally documented; instead, it was distributed on magnetic tape along with a FORTRAN translator that could generate netlists in several other formats. In the contemporary landscape, Verilog has become the most widely used hardware description language. Consequently, translating the ISCAS'85 netlist into Verilog format is essential for seamless integration into the IC design flow. To facilitate this process, a Benchmark Translator, considered a compact Electronic Design Automation (EDA) tool, is required to automatically handle file transformations. Thus, in this programming assignment, **you are requested to implement a C++ program that can take the ISCAS'85 benchmark and translate them to the corresponding gate-level Verilog format**. The resulting Verilog code will then undergo NC-Verilog simulation [5] to validate its correctness.

Background

Benchmarks play a crucial role in ensuring equitable comparisons between different algorithms and CAD tools in the circuit design and optimization processes across various domains, including power, area, performance, and reliability. In the realm of digital designs, several widely recognized benchmark sets are commonly employed by researchers, and they can be accessed on the website [2].

For the conversion of benchmark descriptions into various formats, a compact CAD tool known as the Benchmark Translator is utilized. This tool takes the original benchmark descriptions as inputs and converts them into corresponding formats, such as Verilog, VHDL, Spice, BLIF, and others. An example of implementing the ISCAS'85 benchmarks in VHDL can be found in [3].

Input file format

Your program takes the ISCAS'85 benchmark as input. To illustrate, we will use an example that is a compact six-NAND-gate circuit referred to as "c17", to explain. Figure 1 shows the description file (.netlist) of the c17 circuit. The gate level diagram of the c17 circuit is shown in Figure 2. There are two blocks in the input file. The first block describes the information of each

instance, such as **gate name** and **gate type**. The second block describes the information of each net. The syntax is as following. Note that **inpt** and **outpt** represent the I/O of the module.

```
* Netlist of the module <benchmarkName>

Inst <gateName> <gateType>

NET <netName> <numPins>
PIN <gateName>/<pinName>, ...
```

```
* Netlist of the module c17

Inst gat1 nand
Inst gat2 nand
Inst gat3 nand
Inst gat4 nand
Inst gat5 nand
Inst gat6 nand

NET net1 2
PIN inpt1, gat1/IN1
NET net2 2
PIN inpt2, gat3/IN1
NET net3 3
PIN inpt3, gat1/IN2, gat2/IN1
NET net6 2
PIN inpt6, gat2/IN2
NET net7 2
PIN inpt7, gat4/IN2
NET net10 2
PIN gat1/OUT1, gat5/IN1
NET net11 3
PIN gat2/OUT1, gat3/IN2, gat4/IN1
NET net16 3
PIN gat3/OUT1, gat5/IN2, gat6/IN1
NET net19 2
PIN gat4/OUT1, gat6/IN2
NET net22 2
```

```
PIN outpt22, gat5/OUT1
NET net23 2
PIN outpt23, gat6/OUT1
```

Figure 1: The c17 circuit description (.netlist)

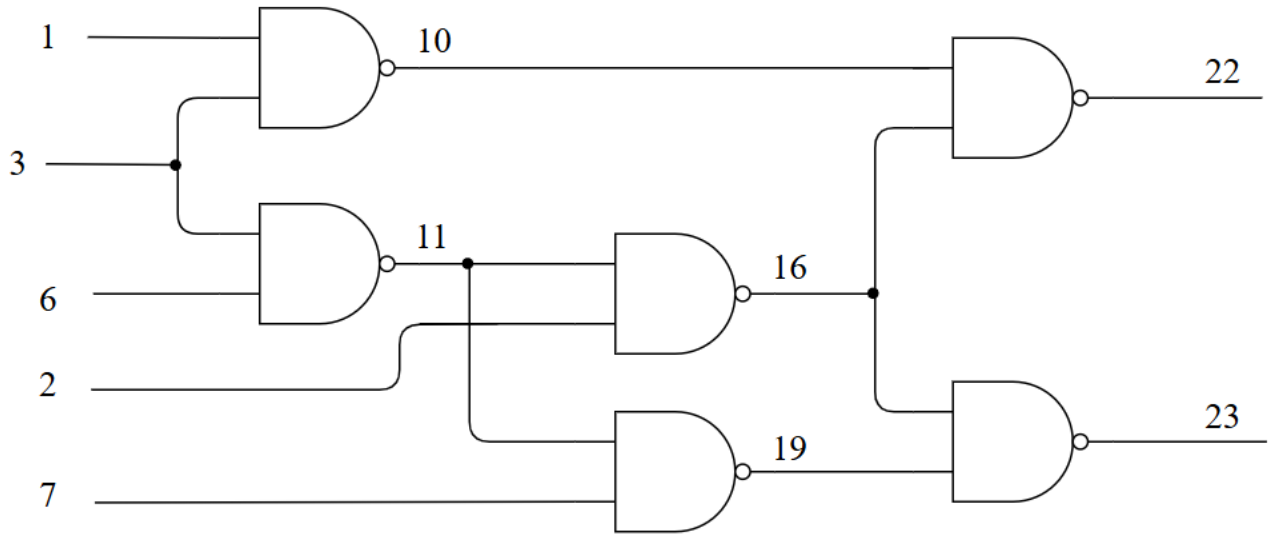

Figure 2: The gate level diagram of the c17 circuit

Output file format

Your program will generate a **synthesizable gate-level Verilog file (.v)**. Figure 3 shows the output of the c17 circuit in Verilog format.

```
`timescale 1ns/1ps
module c17 (N1,N2,N3,N6,N7,N22,N23);
input N1,N2,N3,N6,N7;
output N22,N23;
wire N10,N11,N16,N19;
nand NAND2_1 (N10, N1, N3);
nand NAND2_2 (N11, N3, N6);
nand NAND2_3 (N16, N2, N11);
nand NAND2_4 (N19, N11, N7);
nand NAND2_5 (N22, N10, N16);
nand NAND2_6 (N23, N16, N19);

endmodule
```

Figure 3: The c17 circuit in Verilog format (.v)

The first line starts with "`timescale 1ns/1ps" for simulation purposes. The second line uses "module" as the keyword, followed by the benchmark name. Then, all the inputs and outputs are printed in increasing order with the net name. Then the inputs/outputs are specified. After that, all the wires (internal connections) used in the benchmark are specified. Then the circuit descriptions with different gates are shown. Finally, the keyword "endmodule" appears to identify the ending of the Verilog file. Note that the generated file needs to follow the Verilog Standard of IEEE P1364-2005 [4].

Verification

We will use the **Cadence NC-Verilog simulator** [5] to verify the correctness of your output. A benchmark will be provided for testing the Verilog file you generated. If the generated file can pass the simulation comparison, you will get the point. Two public cases will be provided for your reference, and several hidden cases as well as the two public cases are used to evaluate your program.

Bonus

EDA is a category of software tools and algorithms for designing electronic systems such as integrated circuits and printed circuit boards. Since a modern semiconductor chip may have billions of components, EDA tools are essential for their design; this article in particular describes EDA specifically with respect to integrated circuits (ICs). The tools work together in a design flow that chip designers use to design and analyze entire semiconductor chips. Like Figure 4 below, IC design flow can be briefly separated into four blocks: system, front-end, back-end, and fabrication. Among them, front-end is to uses RTL to design the circuit with the expected function as Figure 5. What's more, Design Compiler® [6] RTL synthesis solution enables users to meet today's design challenges with concurrent optimization of timing, area, power and test.

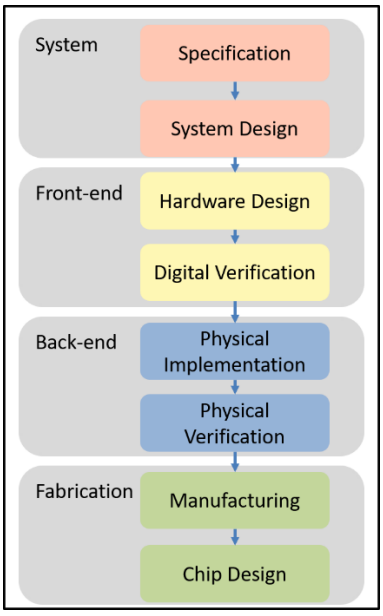

Figure 4: IC design flow

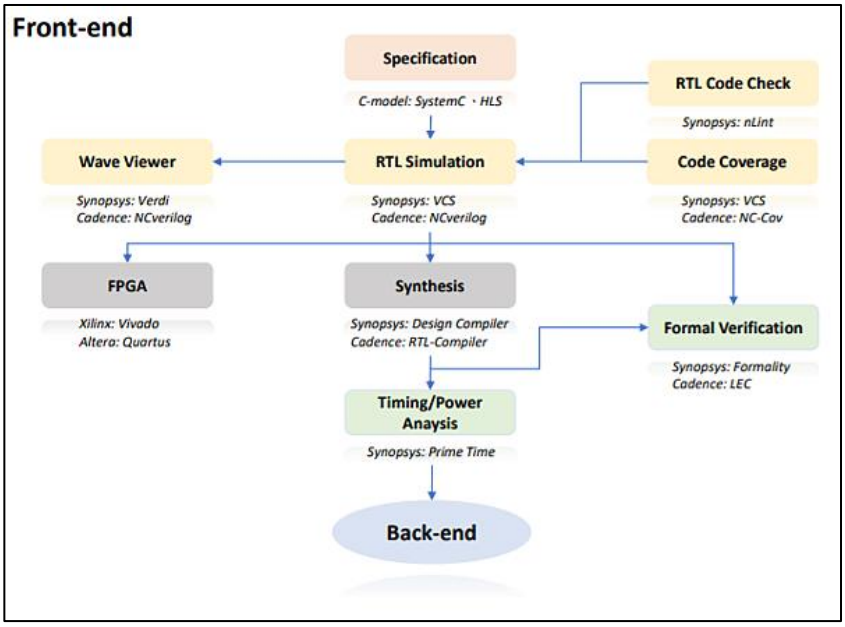

Figure 5: Front-end flow

This bonus exercise provides an opportunity for you to use a commercial EDA tool for front-end design procedure. Design Compiler [6] is the tool that we will use in this exercise. **The goal is to convert the Verilog file you generated into an optimized gate-level netlist mapped to a specific logic library.** More details about the procedure will be provided later. Please follow the slides to complete the bonus portion if you are interested.

Requirement

1. You have to write this program in C or C++. No open-source codes are allowed to be used. (i.e., you MUST implement the tool by yourself). You can use any data structure to realize your program.

2. All files should be submitted through ee-class. The files you need to submit are as follows:
   (1) A source code file named *StudID*_PA1.cpp
   (ex: 9862534_PA1.cpp)
   (2) A report named *StudID_Name*_PA1_report.pdf
   (ex: 9862534_陳聿廣_PA1_report.pdf)

   Note that the only acceptable report file format is .pdf, no .doc/.docx or other files are acceptable. **BE SURE to follow the naming rule mentioned above. Otherwise, 10% penalty will be applied for the file(s) with incompatible name(s).** Note that **duplicating filename extension is considered as an incompatible name.**

3. We will execute your program on a workstation with the following command:
   $ ./StudID_PA1.out inputfilename.netlist outputfilename.v
   (ex: *./9862534_PA1.out c17.netlist c17.v*)
   where the first term is the executable file, the second term is the input file name, and the third term is the output file name.

4. The workstation is configured with GNU GCC Compiler in C++ 11 standard. More details about the workstation will be provided later.

5. We don't restrict the report format and length. In your report, you have to include at least:
   (1) How to compile and execute your program; (You can use screenshot(s) to explain)
   (2) The completion of the assignment; (If you complete all requirements, specify all)
   (3) The hardness of this assignment and how you overcome it;
   (4) Do you have any suggestions about this programming assignment?

Grading

The grading is as follows:

(1) Correctness of your code: 50%
(2) Readability of your code: 10%
(3) The report: 10%

(4) Demo session: 30%

(5) Bonus: 10%

Please submit your assignment on time. Otherwise, the penalty rule will apply:

- Within 24hrs delay: 20% off
- Within 48hrs delay: 40% off
- More than 48hrs: 0 point

Contact

For all questions about PA1, please send email to TA Yin-Rong Zhuo (yinrongzhuo@gmail.com) with the title [CAD PA1].

Reference

[1] David Bryan, "The ISCAS '85 benchmark circuits and netlist format," available online: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=785838

[2] Collection of Digital Design Benchmarks, available online: https://ddd.fit.cvut.cz/www/prj/Benchmarks/index.php?page=download

[3] Neša P. Tomić and Mile K. Stojčev, "ISCAS-85 Netlist Translator into VHDL Code," in ICEST 2004.

[4] IEEE P1364-2005 standard, available on-line: https://www.eg.bucknell.edu/~csci320/2016-fall/wp-content/uploads/2015/08/verilog-std-1364-2005.pdf

[5] Cadence® NC-Verilog® Simulator Help, available online: https://picture.iczhiku.com/resource/eetop/WYKeftERsaRjPmcm.pdf

[6] Design Compiler - Synopsys https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html